SCRIPT-CONTROLLED ACL

Technical Documentation

PROJECT TEAM

Devadharshini.B	**Team Head**
Lakshmi prabha.S	**Project**
Tamildharshini.E	**Voice Over**
Anjali.A	**Documentation**

Script-controlled ACL

Technical documentation

Author	Design owner
Reviewers	Security and platform reviewers
Status	Draft
Date	TBD

1) Summary

Script-controlled access control lists (ACLs) use scripts to define, validate, publish, and evaluate authorization rules. Policy definition is kept separate from runtime enforcement so that rule changes can be reviewed and safely deployed without coupling application logic to a particular platform.

> **Scope note**
>
> The examples in this document are conceptual and framework-neutral. Adapt the data model, execution model, storage, authentication, and deployment procedures to the target platform. No platform-specific APIs, credentials, endpoints, or compliance claims are assumed.

2) Purpose and scope

This design describes a controlled way to manage permissions for subjects requesting actions on resources. It covers policy authoring, validation, versioning, publication, request-time evaluation, auditability, and operational recovery.

- **In scope** — subject and resource matching, actions, effects, conditions, policy lifecycle, enforcement boundaries, and operational controls.
- **Out of scope** — a specific identity provider, policy language, storage engine, API gateway, programming language, or deployment product.

---

3) Key concepts and terminology

- **Subject** — an actor, service, workload, group, or delegated identity requesting access.
- **Resource** — the object, collection, operation target, or capability being protected.

- **Action** — the operation requested, such as read, update, execute, or administer.
- **Rule** — a match expression and decision that produces allow or deny for a request.
- **Condition** — an additional constraint based on trusted request context, such as environment or ownership.
- **Policy** — a versioned, ordered set of rules with metadata, validation status, and publication state.
- **Decision** — the enforcement result, normally allow or deny, plus the rule and policy version used.
- **Default deny** — a request is denied when no valid rule explicitly permits it.

---

4) Architecture and request flow

The architecture has two deliberately separated paths. The control path authors and publishes policy; the data path evaluates an already approved policy during a request. Runtime components must not silently alter policy while evaluating access.

Separation of responsibilities

Policy definition handles authoring, validation, review, versioning, and publication. **Runtime enforcement** authenticates the caller, builds trusted context, evaluates the published policy, returns a decision, and records an audit event.

1. A caller authenticates and submits a request containing a subject, resource, action, and permitted context attributes.
2. The enforcement point verifies that the request context is authentic, complete, and within the expected schema.
3. The enforcement point loads a known published policy version from a trusted source or cache.
4. The evaluator selects applicable rules, resolves precedence, and applies the default-deny behavior when no rule permits access.
5. The enforcement point permits or rejects the operation and records the decision, policy version, and relevant correlation data.
6. Policy changes follow a separate lifecycle: propose, validate, review, publish, observe, and roll back if necessary.

5) ACL data model

An ACL should represent authorization intent without embedding platform-specific storage or execution details. A practical policy record includes the following fields:

- **Policy identity** — stable policy name, version, status, owner, and timestamps.
- **Subject matcher** — a specific identity, group, service, or controlled attribute expression.
- **Resource matcher** — a resource identifier, resource type, or bounded pattern.
- **Action matcher** — one or more normalized actions.
- **Effect** — allow or deny; deny precedence must be explicit and consistent.

- **Conditions** — optional constraints evaluated only against trusted, validated context.
- **Governance metadata** — reason, change reference, reviewer, activation window, and rollback reference.

Subject	Resource	Action	Effect	Condition
team:analytics	report:quarterly	read	allow	environment = production
service:etl	dataset:finance	write	allow	job_owner = service:etl
team:contractors	dataset:finance	write	deny	always
user:example	report:quarterly	administer	deny	always

Evaluation rule

Use a deterministic precedence model, document it, and test it. A common baseline is explicit deny over allow, then the most specific applicable rule, followed by default deny. The target platform may require a different model.

6) Script lifecycle: define, update, validate, and enforce

1. Define rules in a reviewable representation with normalized subjects, resources, actions, effects, conditions, ownership, and version metadata.
2. Update rules through an immutable proposal or a compare-and-change operation. Preserve the prior version and require a reason for the change.
3. Validate syntax, schema, references, condition types, precedence, scope boundaries, and prohibited privilege expansions before publication.
4. Publish only a validated version that has passed the required review and activation checks.
5. Enforce access by evaluating the published version against a trusted request context. Runtime evaluation must be deterministic and side-effect free.
6. Record the policy version and decision inputs needed to explain the result without storing unnecessary sensitive data.

7) Pseudocode examples

```
function check_permission(request, policy):
context = validate_request_context(request)
candidates = select_rules(policy, context.subject, context.resource, context.action)
valid_rules = validate_rules(candidates, context)
decision = evaluate(valid_rules, default = "deny")
record_decision(context, decision, policy.version)
return decision
```

```
function apply_policy_change(change, actor):
require_authenticated(actor)
require_authorized(actor, "policy-write")
normalized = normalize(change)
validate_schema_and_constraints(normalized)
current = read_published_version(normalized.policy_name)
proposed = apply_change(current, normalized)
validate_no_privilege_escalation(current, proposed, actor)
validate_precedence_and_reachability(proposed)
publish_if_version_matches(proposed, current.version)
record_change_audit(actor, current.version, proposed.version)
return proposed.version
```

Adaptation requirement

These examples are conceptual pseudocode. Adapt validation, storage, concurrency control, identity checks, and decision caching to the target platform without weakening the separation between policy management and runtime enforcement.

8) Security safeguards

- **Least privilege** — grant the smallest subject, resource, action, and time scope needed. Separate policy readers, authors, approvers, publishers, and runtime evaluators where practical.
- **Authentication** — authenticate policy operators and request subjects using trusted mechanisms appropriate to the target platform. Reject ambiguous, expired, or unverifiable identity context.
- **Authorization** — authorize policy changes independently from the permissions being changed. Require explicit authority to publish or roll back a policy.
- **Input validation** — validate schema, types, identifiers, condition values, pattern scope, rule count, and maximum evaluation cost before storage or execution.
- **Audit logging** — record who proposed, approved, published, or evaluated a decision; include timestamps, versions, outcomes, and correlation data while minimizing sensitive content.

- **Rollback** — retain immutable prior versions, define a tested rollback action, and make rollback authorization and activation behavior explicit.
- **Idempotency** — give changes stable identities or expected versions so retries do not duplicate rules or produce unintended state transitions.
- **Privilege escalation protection** — compare effective permissions before and after each change, flag expansions for review, prevent self-approval, and reject changes that bypass ownership or scope boundaries.

---

9) Operational guidance

1. **Testing** — maintain unit tests for matching and precedence, negative tests for default deny, regression tests for known incidents, and end-to-end tests for the enforcement boundary.
2. **Deployment** — validate in an isolated environment, review the proposed diff, publish a versioned artifact, use staged activation where supported, and define a rapid rollback path before release.
3. **Monitoring** — track denied and allowed decisions, evaluation errors, stale policy versions, validation failures, latency, cache age, unusual permission expansions, and rollback frequency.
4. **Troubleshooting** — start with the request correlation value, confirm subject and resource normalization, identify the policy version used, inspect matching rules and precedence, then verify deployment and cache state.

Operational invariant

A failure to load, validate, or evaluate a policy should fail closed for protected operations unless the target platform has a separately reviewed emergency behavior.

10) Limitations and alternatives

Script-controlled ACLs provide flexible policy automation, but they introduce evaluation complexity, change-management responsibilities, and a dependency on the correctness of both the policy representation and the enforcement boundary.

Approach	Pros	Cons	Why not the default here
Script-controlled ACL	Flexible, automatable, and adaptable to domain-specific context.	Requires careful validation, versioning, testing, and runtime controls.	Recommended when policy lifecycle and enforcement separation are important.
Static configuration files	Simple to review, reproduce, and deploy.	Limited dynamic context and slower response to frequent changes.	May be sufficient for small, stable permission sets.
Central policy decision service	Consistent evaluation and centralized governance across applications.	Adds service availability, latency, integration, and operational dependencies.	Prefer when many consumers need shared policy decisions.
Application-level checks	Close to business logic and easy to start with.	Risk of duplicated, inconsistent, or bypassable authorization logic.	Use only when the enforcement boundary is controlled and well tested.

11) Open questions

- Which identities, resources, actions, and trusted context attributes must the target platform support?
- What precedence model and default behavior are required for conflicting rules?
- Which roles may propose, approve, publish, and roll back policy versions?
- What consistency, availability, latency, and cache-staleness limits apply to runtime enforcement?
- How long should policy versions and decision records be retained, and who may inspect them?
- Which permission expansions require a second reviewer or an explicit migration window?

Implementation checklist

- ☐ Define the policy schema, normalized vocabulary, precedence model, and default-deny behavior.
- ☐ Document the boundary between policy management and runtime enforcement.

- [ ] Implement authentication and separate authorization for policy operations and protected resources.
- [ ] Add schema, scope, reachability, evaluation-cost, and privilege-expansion validation.
- [ ] Store immutable versions with stable change identities and rollback references.
- [ ] Create positive, negative, regression, concurrency, failure-mode, and rollback tests.
- [ ] Define deployment, monitoring, alerting, troubleshooting, and incident procedures.
- [ ] Adapt all conceptual examples to the target platform and complete security review before production use.